SR-MPLS MTU: Packet Size, Label Depth, and Path Design

How to size every link for reliable, scalable Segment Routing

Author: ENAM AHMED **@ThePacketPathWithPartha**

Abstract

Segment Routing over MPLS (SR-MPLS) carries segment instructions in an MPLS label stack. Each stack entry adds four bytes, and the number of labels can change as the packet moves through the network. A link can therefore pass ordinary 1,500-byte IP traffic but fail when SR-MPLS or a service adds labels. This paper explains the different meanings of MTU, shows how to calculate label overhead, distinguishes byte MTU from label-stack depth, and gives practical design and testing steps. The examples use common Ethernet and IP values only as illustrations; exact MTU accounting depends on platform and interface.

1. Introduction

A network's MTU is the largest packet or frame that an interface can forward. If a packet is too large for one link, the network must either signal a smaller size, fragment the packet where permitted, or drop it. The result can be confusing: small pings work, but large transfers stall.

SR-MPLS adds a label stack in front of the IP packet. Each MPLS label-stack entry is four bytes. The SR architecture uses these labels as instructions, and an SR Policy can use several labels to guide a packet along a chosen path. The extra bytes are small, but the stack can grow when the policy is more detailed or additional services are carried. [1] [2] [3]

The design question is not simply, "Is the core MTU 1,500 or 9,000?" The useful questions are: What is the largest complete IP packet the service must carry? How many labels can be present on each link? What exactly does this platform's MTU setting count? And does every normal and failure path support both the byte size and the label depth?

@ThePacketPathWithPartha

2. Understand the MTU terms

Engineers often use "MTU" for different measurements. The value `1500` is not meaningful until its starting point is known.

Term	What the size normally includes
IP MTU	The complete IP packet, starting at the IP header. It includes the IP header and IP data, but not the Ethernet header.
MPLS packet size	The MPLS labels plus the packet underneath them. Each label adds four bytes. Vendor commands may call the related limit MPLS MTU.
Layer 2 (L2) or frame size	The link-layer frame as counted by that platform. It may include an Ethernet header and may treat VLAN tags or other encapsulations differently.
Path MTU (PMTU)	The maximum IP packet that can travel from a source to a destination over the path in use. The smallest effective limit on that path is the bottleneck.

Table 1: Use the platform's definition, not just the displayed MTU number.

A 1,500-byte IP MTU means the entire IP packet is 1,500 bytes. It does not mean 1,500 bytes of application data. Likewise, a Cisco IOS XR guide describes L2 MTU as starting at the Ethernet header, IP MTU as starting at the IP header, and MPLS MTU as starting at the MPLS labels. Its documented example adds the label bytes and a 14-byte Ethernet header separately. Other platforms may use different names or counting rules, so check the hardware and release documentation. [8]

VLAN tags, QinQ, pseudowire headers, tunnels, and other link encapsulations can add bytes on the wire. Add them only when they fall inside the specific size limit you are configuring. Avoid two common errors: forgetting overhead that a port actually counts, and adding the same Ethernet header twice.

3. How the SR-MPLS stack changes packet size

An MPLS label-stack entry is 32 bits, or four bytes. The entry contains a 20-bit label value, a 3-bit Traffic Class field, a 1-bit Bottom-of-Stack flag, and an 8-bit TTL. The 20-bit label value may be a large number, but it still occupies one fixed-size four-byte entry. The final entry has the Bottom-of-Stack flag set; it marks where the IP packet or another payload begins. [1]

In SR-MPLS, the top label is the active segment. The router processes that segment. When the segment is completed, its label is popped, and the next label becomes active. This means the stack is not always the same depth on every link. A Node-SID may be swapped along a route, an Adj-SID may be consumed at its owner, and penultimate-hop popping (PHP) may remove a label before the egress. Measure the actual stack on the link being sized. [2] [3]

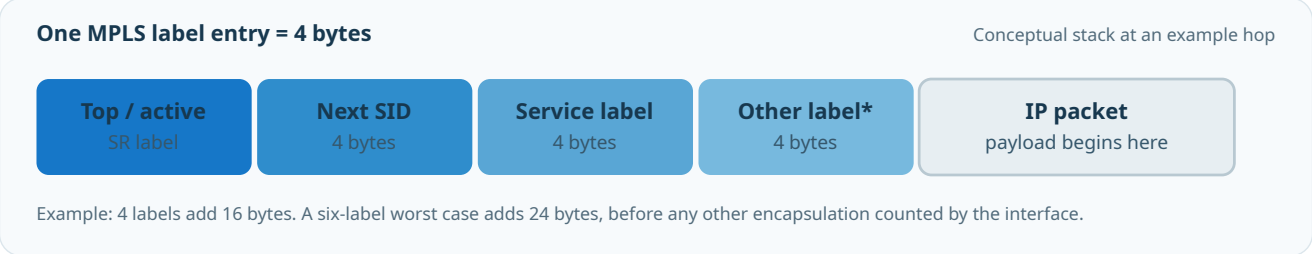


Figure 1: A conceptual SR-MPLS label stack. Every MPLS entry contributes four bytes, regardless of its label value.

The labels in a packet depend on the service and policy. A basic SR-MPLS Layer 3 VPN packet often has a transport or Node-SID label plus a VPN/service label. A longer SR Policy can add more segment labels. Entropy Label Indicator (ELI) and Entropy Label (EL), if used, add two more entries. An SR Policy's Binding SID can represent the policy at a node, but the policy's segment list may be imposed or expanded at that node. Therefore, a Binding SID can change where stack depth occurs; it does not remove the need to check each link. [3] [5]

4. Calculate the byte budget

Use the same packet-size reference point throughout the calculation. For each link, first count the MPLS labels that are actually present there.

> **MPLS packet size on a link = complete IP packet size + (4 bytes × MPLS labels present on that link)**
> **Required L2/frame size = MPLS packet size + other link encapsulation bytes included by that interface's MTU definition.**

For a 1,500-byte IP packet, illustrative stacks give these values:

Stack example	Labels	Label bytes	MPLS size
One transport/SR label + one service label	2	8 B	1,508 B
Four SR segment labels + one service label	5	20 B	1,520 B
The same five labels + ELI and EL	7	28 B	1,528 B

Table 2: Count only labels that coexist in the stack at the link being checked.

These are examples, not fixed SR-MPLS recipes. A policy may use a different number of SIDs. Some labels may have been removed by the time the packet reaches a later link. Do not add a separate “transport label” if the active SR label already provides that transport function. The correct count is the number of actual four-byte entries in the packet at that hop. If using the RFC 6790 ELI-plus-EL method, add two entries, or eight bytes, at the points where that pair is present. [5]

For example, if a 1,500-byte IP packet carries six MPLS labels, its MPLS size is 1,500 + (6 × 4) = 1,524 bytes from the first label to the end of the IP packet. Under the specific IOS XR L2 accounting example, adding a 14-byte Ethernet header gives 1,538 bytes before any additional counted encapsulation. Do not treat 1,538 as a universal command value. Check whether your platform includes VLAN tags, frame check sequence, or other bytes in its reported MTU. [8]

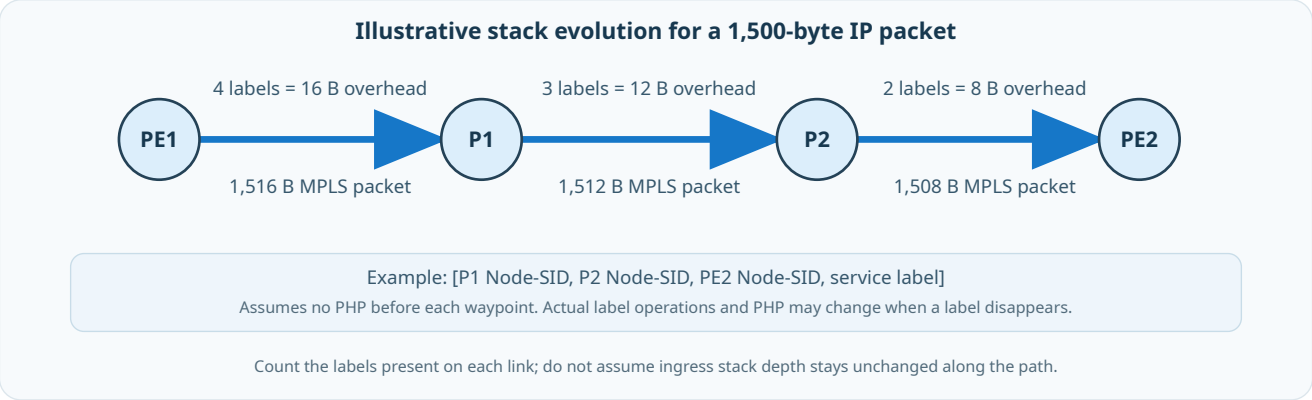


Figure 2: A simple path example showing how the stack can shrink as segments complete. The values are illustrative; actual PHP and label operations matter.

5. MTU and MSD are separate limits

A packet can fit within the byte MTU and still fail if a router cannot process that many labels. The reverse can also happen: a router may support the number of labels, but the packet plus those labels may exceed the link MTU. Both tests must pass.

Maximum SID Depth (MSD) is a signaled stack-depth capability for a node or link. RFC 8491 defines Base MPLS Imposition MSD (BMI-MSD) for IS-IS. BMI-MSD counts the total MPLS labels that can be imposed, including service, transport, and special labels. A node-level value represents the smallest supported value across the interfaces in that IGP instance; a link-level advertisement can describe a specific link. If a controller or path-computation function uses MSD, it can avoid paths that require more labels than the participating hardware supports. Advertisements are optional and platform support varies, so check the actual values rather than assuming they exist. [4]

Use two checks for every relevant path. First ask, “Are there enough bytes on every link for the IP packet, labels, and counted encapsulation?” Then ask, “Can every router that must impose the stack support that label depth?” A large configured MTU does not prove the label-depth limit is sufficient. A high MSD does not increase the interface MTU.

6. See the bottleneck in a topology

The topology below shows a preferred SR policy path and a possible alternate or repair path. The links may have different MTUs, and the policy may put a different stack on the alternate route. If a 1,500-byte customer IP packet needs 1,524 bytes from the MPLS label stack to the packet end, a link that can carry only 1,520 bytes becomes a bottleneck. The preferred path can work while the backup path drops the same traffic after a failure.

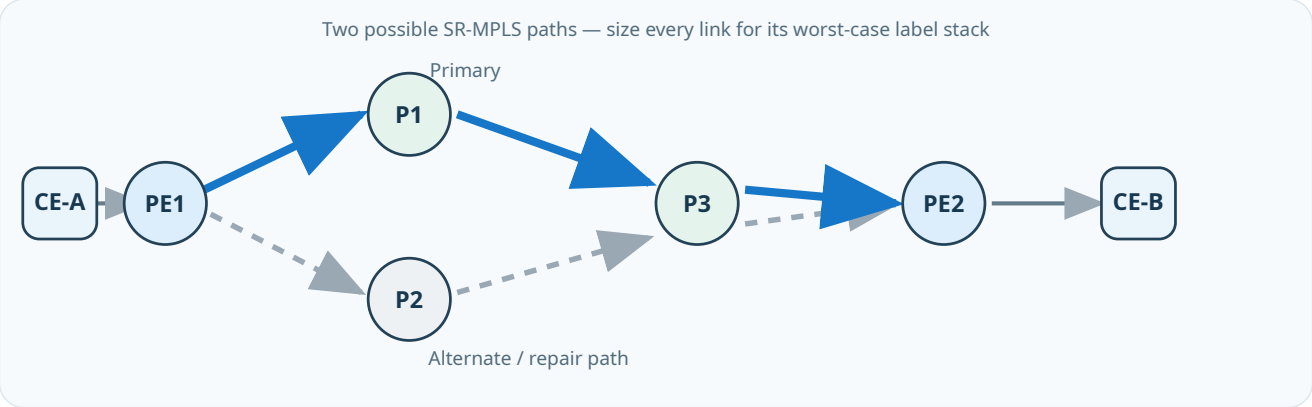


Figure 3: A path is only as strong as its smallest effective MTU and label-depth limit. Check every ECMP and repair path.

An end-to-end service should be sized for its most demanding traffic and its largest relevant stack on every eligible path. The number of hops alone does not add MPLS label overhead; the labels present on a packet at each hop do. However, routing changes, ECMP selection, PHP, and policy expansion can change which link is the limiting one.

7. What happens when the size is wrong?

Small control packets may still pass because they are far below the limit. Large packets expose the mismatch. Users may report that a TCP connection starts but a file transfer or web page stops partway through. A small ping may pass while a larger one fails. With ECMP, only traffic using one member may fail. A failure can also reveal a bad backup path that was not used during normal operation.

For IPv4, Path MTU Discovery commonly sends packets with the Don't Fragment (DF) bit set. If a router cannot forward one, it drops the packet and returns an ICMP Destination Unreachable message meaning "fragmentation needed and DF set," with the next-hop MTU when supported. If DF is clear, a router may fragment in some circumstances, but MPLS behavior and platform implementation vary. Fragmentation increases overhead and should not be used as the normal design. [1] [6]

IPv6 routers do not fragment packets in transit. A router that cannot forward an IPv6 packet should drop it and send an ICMPv6 Packet Too Big message so the source can send smaller packets. If that feedback is blocked or not delivered, a connection can appear to hang when larger data starts, even though the initial handshake succeeded. Allow required ICMP feedback through security policy; do not block it indiscriminately. [1] [7]

An MPLS router may need to remove labels to generate an IP-layer error and may need a way to send that error back to the original source. The exact exception path, rate limits, and message behavior are implementation-specific. This is why a working PMTU message is helpful, but not a substitute for configuring enough MTU on every link. [1] [8]

@ThePacketPathWithPartha

8. A scalable design and rollout method

1. **Define the service target.** Write down the largest complete IPv4 or IPv6 packet the service must carry. Include the IP header. If a customer requirement is stated as application bytes, convert it to an IP packet size first.
2. **Build a label inventory.** For every service and SR Policy, record the maximum label stack expected at each relevant hop. Include transport and service labels, all SIDs in a policy, optional ELI/EL, and any nested MPLS encapsulation. Include the stack that can appear on a repair path. Do not double-count labels that perform the same role.
3. **Check the packet-size budget.** On each link, add four bytes for every MPLS label present to the complete IP packet size. Then add Ethernet or tunnel overhead only according to that interface's documented MTU definition.
4. **Check the depth budget.** Compare the required stack with the hardware's imposed and forwarded label-depth limits. Where supported, verify node/link BMI-MSD advertisements and confirm that the policy controller or path-computation function uses them correctly.
5. **Cover every path.** Check both directions if routing can be asymmetric. Check every ECMP member, bundle member, inter-area link, SR policy, and protection path. A uniform core MTU simplifies operations, but the configured value still has to fit the hardware and transport.
6. **Plan for growth.** Leave intentional room for approved new services, additional SIDs, or encapsulation changes. Do not use a blanket "add 100 bytes" rule without knowing the future stack and accounting boundary. Recalculate when policies, labels, transport, or hardware change.
7. **Document the result.** Record the customer packet size, maximum per-link label count, required MPLS and L2 sizes, platform-specific meanings, MSD values, and validation evidence. This gives the next engineer a calculation they can repeat.

@ThePacketPathWithPartha

9. Test and troubleshoot safely

First inspect the configured and operational MTUs on the full path. Include physical interfaces, subinterfaces, port-channel or bundle members, and the corresponding MPLS interface values. Confirm the maximum stack in forwarding state or a packet capture. Check interface drops, oversized-packet exceptions, fragmentation counters, and ICMP/ICMPv6 messages.

Then use size-controlled probes from the service endpoints. The size entered into a ping command may be the ICMP data size, not the complete IP packet. For a basic 1,500-byte IPv4 ICMP echo packet with no IP options, the arithmetic is 20 bytes IPv4 header + 8 bytes ICMP header + 1,472 bytes echo data = 1,500 bytes. For IPv6, 40 bytes IPv6 header + 8 bytes ICMPv6 header + 1,452 bytes echo data = 1,500 bytes. Extension headers or IP options add more bytes and must be included in the complete packet size. Tool syntax varies, so check whether its size option means payload or total packet.

Probe the largest size required by the service, then test smaller sizes around the failure point. Test IPv4 with DF behavior and test IPv6 Packet Too Big feedback where the platform allows. Repeat over the preferred path, alternate ECMP paths, and protection path. A ping to a router loopback may follow a different path or use a different label stack from customer traffic, so also test the actual service and application flow.

10. Common design mistakes

A frequent mistake is treating `1500` as application payload rather than a complete IP packet. Another is counting only SR SIDs while forgetting a VPN/service label or an optional ELI/EL pair. Teams also sometimes use the same MTU number across vendors without checking whether it starts at the Ethernet header, MPLS label, or IP header.

Do not assume that a single successful ping proves every path. Do not assume the backup route has the same MTU as the preferred route. Do not assume an advertised MSD is present, correct, or being used by the controller. Finally, do not assume that increasing MTU alone fixes the issue: the forwarding hardware must also support the label depth, and the service endpoints must still agree on their IP MTU.

Conclusion

MTU is essential to SR-MPLS because the label stack adds bytes to packets and because stack depth can vary across services, policies, and paths. A reliable design starts with the largest complete IP packet, counts the actual MPLS labels on every link, follows each platform's MTU accounting, and checks label depth separately. Validate normal, ECMP, and failure paths with real service traffic. When these steps are documented and repeated after changes, the network can grow without turning a small MTU mismatch into a large outage.

References

[1] RFC 3032, **MPLS Label Stack Encoding**. Label-entry format and oversized labeled packet handling.

[2] RFC 8660, **Segment Routing with the MPLS Data Plane**. SR-MPLS forwarding behavior.

[3] RFC 8402, **Segment Routing Architecture**. SR Policies, active segments, and Binding SIDs.

[4] RFC 8491, **Signaling Maximum SID Depth (MSD) Using IS-IS**. Node/link MSD and Base MPLS Imposition MSD.

[5] RFC 6790, **MPLS Entropy Labels**. The ELI-plus-EL stack increase.

[6] RFC 1191, **Path MTU Discovery**. IPv4 DF and fragmentation-needed feedback.

[7] RFC 8201, **Path MTU Discovery for IP version 6**. IPv6 Packet Too Big feedback and PMTU behavior.

[8] Cisco IOS XR 8000, **Configuring MPLS MTU**. Platform-specific MTU definitions, sizing example, and packet handling.

@ThePacketPathWithPartha